

Programmable In-Network Timing Obfuscation for DNP3

Anonymous Submission

Abstract—Adversaries preparing attacks on industrial control systems (ICSs) such as power grids first perform reconnaissance, and device fingerprinting supplies much of the knowledge they need. In ICS networks, the fingerprint does not have to come from the payload. The interval between the transport acknowledgment of an outstation and its application response carries the execution time of the device, and it identifies both the device and the operation it performs. Suppressing that timing variation is challenging when legacy endpoints must continue exchanging readable protocol messages. We focus on changing the timing of those existing exchanges.

To address this problem, we propose an in-network timing-obfuscation framework that schedules acknowledgments and responses around an interval the operator configures. A programmable switch in front of the outstation holds the acknowledgment and the response and releases each of them at a deadline computed from the arrival of the request. Packets arriving before their deadlines can follow this schedule, subject to queue service and the supported holding range. The switch only delays the packets it receives, so both endpoints run unmodified and no DNP3 byte changes. We implement the framework on an Intel Tofino-1 switch and evaluate it with a physical SEL-751A protective relay over 63,360 DNP3 exchanges. The experimental results show that the measured CLRT medians approach the configured 8 ms for READ, SELECT and OPERATE, while the READ interquartile range falls from 2.772 to 0.022 ms. A classifier trained before deployment falls from 0.803 balanced accuracy to chance, although an adaptive classifier retains accuracy of 0.445. The median READ request-to-response time increases to 28.107 ms, with no retransmission observed in the campaign.

Index Terms—traffic obfuscation, device fingerprinting, DNP3, programmable switches.

I. INTRODUCTION

Device fingerprinting has been an essential step in cyber reconnaissance, allowing adversaries to reveal unique features of target networks and to design effective, stealthy attack strategies. These techniques are becoming increasingly critical in industrial control systems (ICSs) such as power grids, where adversaries often use IP-based control networks and computing devices within as entry points to inflict physical damage. Consequently, fingerprinting shifts the focus from visited websites and user biometric behavior to device models and types of control operations that are critical to ICS attacks. In the 2015 attack that disrupted Ukrainian power grids and the

Stuxnet attack that disrupted Iranian nuclear power facilities, it is widely believed that adversaries stayed in their systems for at least 6 months to perform cyber reconnaissance [1], [2].

To disrupt device fingerprinting, many studies have presented network traffic obfuscation [3], [4]. Device fingerprinting targeting general computing environments generally relies on network-level features such as packet size and/or inter-packet latency observed from communication patterns [5]. Consequently, existing traffic obfuscation often focuses on (i) padding and splitting network packets, which hide or change the distribution of network packet sizes [3], [6], [7], and (ii) delaying network packets and adding dummy ones, which disrupt the inter-packet timing pattern [4], [8]–[10]. Since manipulating communication networks can introduce runtime overhead, recent works have begun to offload traffic obfuscation onto programmable network switches, which change communication patterns at much higher line rates than CPUs [11]–[13].

Unfortunately, these methods cannot be directly applied to ICS environments for two main reasons. First, device fingerprinting methods on ICS devices rely on different features from the ones in general computing environments. Instead of using network layer knowledge, they attempt to use behavior on physical devices to reveal device model or types of operations. For example, work in [14]–[17] uses the latency between the TCP acknowledge packets and the actual response from the same device to estimate execution time there. Since each vendors may choose different materials or algorithms to implement the same control functionality, this time can be accurate in fingerprinting those devices. Second, existing traffic obfuscation is performed over encrypted channels, which are necessary to mix the obfuscated and normal traffic. Unfortunately, ICS networks still rely on unencrypted traffic due to a wide range of legacy devices, despite ICS network protocols may provide security features. Directly padding or adding dummy packets can easily reveal obfuscated packets, exposing the the underlying traffic pattern to adversaries.

In this paper, we propose an in-network timing-obfuscation framework for ICS traffic. Instead of padding packets or adding dummy ones, the framework changes only when the packets of a transaction leave the network. It schedules the acknowledgment and response from the request, so packets that arrive in time can follow an operator-configured interval, subject to release effects.

A. Contributions

To overcome the two drawbacks above, we design a framework that runs on a programmable switch placed in front of an outstation. The switch holds the transport acknowledgment and the application response of the outstation, and it releases each of them at a deadline computed from the moment the request arrives. Because both deadlines are computed from the request, their scheduled values do not depend on when the outstation acknowledges or answers. Actual release also depends on packet availability and queue service. The framework works with readable protocol traffic by holding existing packets rather than requiring endpoint encryption.

The Framework's Role. We position the framework as a complement to the protections an ICS already runs, such as intrusion detection. The framework cannot hide which operation an exchange carries, because DNP3 leaves the function code readable, and the release budget controls the scheduled delay of protected exchanges. It leaves the application payload unchanged and requires no modification of the master, the outstation or the DNP3 protocol. Consequently, it can protect legacy devices that can neither be patched nor replaced, which are the devices that fingerprinting targets.

Even though we present the framework for DNP3, the design concept can be extended to other ICS protocols. The scheduling idea requires an identifiable request and returning acknowledgment and response. Applying it to another protocol would also require parsing, transaction matching and timer checks for that protocol, which we do not evaluate here.

The main contributions of the paper are:

- **Replace arrival-dependent timing with scheduled intervals.** We design a release schedule that holds real protocol packets and computes both release deadlines from the arrival of the request. The operator sets the scheduled request-to-acknowledgment interval and CLRT, while packet availability and residual release effects constrain how closely actual timing follows.
- **Obfuscate plaintext ICS traffic without adding a packet.** The framework only delays packets and requires no endpoint encryption. We relate the release budget to packet availability and protocol timers, then evaluate the delay and observed timer behavior.
- **Implement the framework at line rate.** On an Intel Tofino-1 switch, we hold each packet behind a strict-priority queue of blocker packets the switch generates. The queues sustain the wait while P4 checks deadlines on returning blockers. We cover both the read and the control transaction of DNP3.

For evaluation, we placed the switch between a DNP3 master and a physical SEL-751A feeder protection relay and collected 63,360 exchanges in 22 paired runs, each containing both cases. We recorded packet timestamps at the master-facing link, and we tested it against (i) a classifier trained before the framework was deployed, (ii) an adaptive adversary that retrains on obfuscated traffic, and (iii) an adaptive

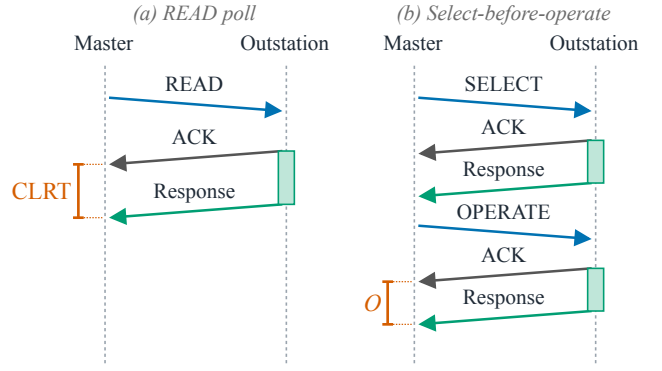


Fig. 1. The two DNP3 transactions on the master-facing link. (a) A READ poll and its CLRT. (b) A select-before-operate control and its master-visible interval O .

adversary that pools many exchanges of the same operation into one signature, as Formby et al. do.

Our experimental results show reduced timing variation and fixed-classifier accuracy, together with the latency cost and residual information:

- The released interval settles on the configured value. The medians of READ, SELECT and OPERATE move from 2.121, 2.041 and 2.929 ms to 8.005, 8.005 and 8.000 ms, and the interquartile range of READ falls from 2.772 to 0.022 ms. The operator can place the interval anywhere from 2 to 26 ms within a fixed budget, and the measured median follows each tested setting to within 6 μ s.
- Timing-based separation decreases. A classifier that separates READ and SELECT at 0.785 balanced accuracy under Timing OFF is at 0.507 under the framework, against a chance level of 0.5. A classifier trained before deployment falls from 0.803 to 0.333 on the three operations, i.e., chance. An adaptive adversary reaches 0.445, and the residual tests find timing information associated with request spacing without isolating its cause.
- The latency cost is measured. Median READ request-to-response time rises from 2.681 to 28.107 ms. No TCP retransmission was observed, and the longest request-to-ACK wait was 20.19 ms, compared with the master's separately observed 200.8 ms retransmission interval.

II. BACKGROUND AND MOTIVATION

In this section, we first present the DNP3 transactions the framework protects. Then we explain why the timing of these transactions identifies a device and how a programmable switch can change that timing.

A. DNP3 Read and Control Transactions

DNP3 carries supervisory control and data acquisition traffic over TCP between a master and an outstation [18]. In Figure 1, we show the two transactions the framework protects. The master polls the outstation with a READ request, function code 1, and the outstation returns the requested measurements in a response, function code 0x81. A control command instead uses

select-before-operate. The master sends a SELECT, function code 3, that names an output point, and the outstation confirms it. The master then sends an OPERATE, function code 4, which the outstation executes and confirms with a solicited response.

Because DNP3 runs over TCP, the transport layer of the outstation acknowledges each request. The acknowledgment travels in a segment of its own before the application answers, although TCP also allows it to ride on the response [19]. Each exchange appears on the wire as three packets: a request, a transport acknowledgment carrying no DNP3 payload, and a DNP3 response. We limit the framework to the three function codes above, READ, SELECT and OPERATE, and it forwards every other exchange unchanged.

Definition 1. Cross-Layer Response Time (CLRT): the interval between the arrival of the transport acknowledgment and the arrival of the DNP3 response of the same exchange, measured on the link to the master.

Definition 2. Physical Operation Time: for a select-before-operate command, the interval between the OPERATE command and the moment the commanded physical change occurs at the device. Formby et al. estimate it in two ways, from the arrival of the unsolicited event response at a tap and from the sequence-of-events timestamp the response carries in its payload [14]. They report that the arrival-time estimate produced no usable results and retain the payload timestamp as the physical fingerprint.

B. Execution Time as a Fingerprint

An outstation answers a request twice, from two different layers. Its TCP stack acknowledges the request as soon as it arrives, whereas its application responds only after the device has done the requested work. Because both packets travel the same path in the same direction, the common propagation delay cancels. Consequently, the CLRT is dominated by the execution time of the outstation.

Execution time differs between device models, since vendors implement the same function with different hardware and algorithms, and it differs between the operations one device performs. Formby et al. classify DNP3 device types and models by the distribution of the CLRT [14], and Lin et al. measured the execution times of three devices, among them the SEL-751A we evaluate, and separated them by the combination of their average and their variation [20]. The same separation holds between the operations of one device. For example, the median CLRT of our relay is 2.929 ms for a control command and 2.121 ms for a poll, while its median acknowledgment latencies for READ, SELECT and OPERATE lie within 0.003 ms of one another. Consequently, the CLRT carries how this device performs an operation, which is the profile a fingerprinting adversary builds.

C. Holding Packets in a Programmable Switch

A programmable switch exposes a match-action pipeline, written in P4 [21], that parses headers, keeps state in registers and places packets into output queues at line rate. Each egress

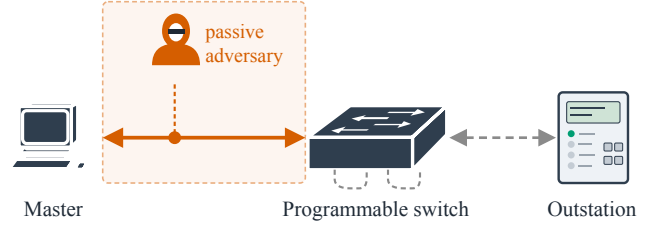


Fig. 2. Observation model. The adversary monitors the shaded master-facing link. The dashed paths are not visible to it.

port offers several queues served in strict priority, a primitive that can also approximate programmable scheduling [22]. Because such a pipeline can delay a packet and release it later without a host in the loop, a switch between the master and the outstation can hold the acknowledgment and the response of a transaction and release them on a schedule the operator sets. As such, the switch can change the timing an adversary observes without changing either endpoint or any byte of the payload.

III. THREAT MODEL AND OBJECTIVES

In this section, we first describe the adversary we consider and the knowledge it seeks. Then we specify the three research objectives the framework must achieve, RO1 to RO3, and the scope of our work.

A. Threat Model

Assumptions on Adversary's Capability. We assume that an adversary has penetrated the control network and passively monitors the master-facing link, the position from which Formby et al. fingerprint outstations [14], as shown in Figure 2. The adversary is able to record the headers, the size and the arrival time of every packet. Because DNP3 carries no transport security by default [23], the adversary also reads the function code of every request and can label each exchange by its operation, e.g., a READ poll or the SELECT and OPERATE of a control command. We assume that the adversary knows the framework and the interval it enforces, but cannot read the internal state of the switch or monitor the link between the switch and the outstation. This is a practical assumption, as the switch sits at the edge of the outstation, and the relay-facing link can be a single cable between two devices in the same substation cabinet.

Adversary's Goal. The adversary aims at fingerprinting the outstation from the timing of its exchanges, and from that timing it infers the make and model of the device and the operation each exchange performs. Based on this knowledge, the adversary can design targeted attacks, e.g., false data injection that must match the devices it compromises [24]. We consider two adversaries. A *fixed* adversary builds its classifier from traffic captured before the framework is deployed and applies it afterwards. An *adaptive* adversary collects traffic after deployment, labels it with the plaintext function codes and retrain its classifier. The adaptive adversary can also pool

many exchanges of the same operation into one signature, as Formby et al. do [14].

B. Research Objectives

The Framework's Objective. We aim at disrupting timing-based fingerprinting of an outstation without changing the outstation, the master or the DNP3 protocol. We specify three research objectives (RO) as follows:

- **RO1:** for both the read poll and the select-before-operate control, we aim at making the CLRT the adversary measures a value the operator configures, independent of the execution time of the outstation.
- **RO2:** for fixed and adaptive adversaries, we aim at removing the execution time of the outstation from everything they learn from the timing of an exchange, such that a fixed adversary performs no better than chance and whatever an adaptive adversary still learns reflects the switch rather than the device.
- **RO3:** for the operation of the control network, we aim to keep added delay within the endpoint timer margins without adding packets or bytes to the external exchange. We measure the latency cost and check for retransmissions and refused control operations under the evaluated settings; these observations do not establish a timer guarantee for other deployments.

Trusted Components. To achieve these objectives, we add only the switch and its control plane to the trusted computing base. The control plane installs the release policy and reads it back, and the switch enforces it. The framework needs no cooperation from the master or the outstation, and neither of them is modified.

For Payload-Based Fingerprinting. We designed the framework to hide timing, not content. It forwards every packet at its original size and leaves the function codes readable, so the adversary still knows which operation an exchange carries. The framework aims to suppress the execution-time variation carried by the released intervals. The physical operation time of Definition 2 is a second feature, and the estimate Formby et al. retain reads a timestamp inside the payload rather than the timing of a packet [14]. Such content is a different problem, and payload protection is complementary to the framework. We leave the combination of the two to future work.

For Active Adversaries. An adversary that injects, delays or drops packets, or that compromises the switch or the outstation, performs attacks beyond reconnaissance and is outside this model. Such an adversary still faces existing defenses, e.g., specification-based intrusion detection for DNP3 [25], and the framework leaves every byte those defenses inspect unchanged.

IV. DESIGN

In this section, we first describe the timing model and the packet flow through the framework. Then we define the design parameters and the conditions under which the switch can follow the release schedule. Last, we explain how queues and the P4 pipeline realize this schedule.

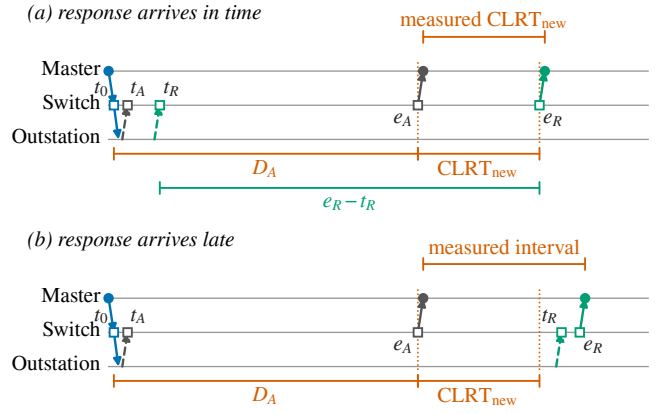


Fig. 3. Release timeline of one READ transaction. All five instants are at the switch: the request arrives at t_0 , the acknowledgment and response arrive at t_A and t_R , and the switch releases them at e_A and e_R . The schedule is anchored at t_0 . (a) The response arrives before its deadline. (b) It arrives late and leaves when its queue next serves it. The drawing illustrates the ideal schedule; actual release also includes queue service.

A. Timing Model and Packet Flow

To change the timing an adversary observes, we place a programmable switch between the master and the outstation. As a READ poll or the SELECT phase passes through the switch, we forward the request and hold the returning acknowledgment and response, which carry the timing seen by the adversary. For OPERATE, it also holds the command before forwarding it to the relay. Because both cases use the request's arrival to schedule the returning packets, the release deadlines do not follow the outstation's acknowledgment latency.

Timing Model. In Figure 3, we show the timing of a READ transaction. We denote the request's arrival at the switch by t_0 , the acknowledgment and response arrivals by t_A and t_R , and their departures toward the master by e_A and e_R . At this switch boundary, the original acknowledgment-to-response interval is

$$\text{CLRT}_{\text{original}} = t_R - t_A. \quad (1)$$

Holding the two packets changes that interval to

$$e_R - e_A = \text{CLRT}_{\text{original}} + (e_R - t_R) - (e_A - t_A), \quad (2)$$

where $e_A - t_A$ and $e_R - t_R$ are the acknowledgment and response holds. The adversary measures CLRT on the master-facing link, where the difference in path and capture delay between the two packets also contributes to the interval. We distinguish that measurement from the switch-side interval when interpreting the results.

Replacing the Interval. Because a constant hold on each packet shifts the original CLRT by the difference between the holds, it preserves the variation that a timing classifier can use. To absorb that variation, we choose release deadlines from the request, such that a packet arriving early waits longer and one arriving closer to its deadline waits less. This choice requires both packets to arrive before their respective deadlines, which constrains the parameters below.

Holding with Queues. To keep a packet waiting, we place it in a queue below another queue of higher priority. The higher queue contains internal *blocker packets*, which the scheduler serves instead of the held packet for as long as the higher queue remains nonempty. We circulate the blockers until their deadline, after which they stop returning. After the remaining blockers drain, the scheduler can serve the held packet. In this way, the queues provide the wait, while the P4 program decides which packets enter them and when blockers stop.

B. Design Parameters and Constraints

Release Deadlines. We use two configured parameters: D_A , the acknowledgment deadline measured from the request, and the configured CLRT_{new} , the desired gap between the acknowledgment and response. When a request arrives, we set

$$\hat{e}_A = t_0 + D_A, \quad \hat{e}_R = t_0 + D_A + \text{CLRT}_{\text{new}}. \quad (3)$$

The hats denote scheduled instants, not measured departures. We call their combined offset

$$D = D_A + \text{CLRT}_{\text{new}} \quad (4)$$

the response *release budget*. For example, the evaluated configuration uses $D_A = 20$ ms and a configured CLRT of 8 ms, giving $D = 28$ ms. Note that D_A starts at the request, so part of this interval has already elapsed when the acknowledgment reaches the switch; its hold is only the remaining wait.

Per-Transaction Policies. A fixed policy uses the same offsets on every admitted request. We can also select a pair of offsets per request from a configured distribution. The selection does not use the operation code, and the chosen offsets remain fixed for that transaction. The availability conditions below apply to every selected pair.

Packet Availability. Because the switch cannot release a packet it has not received, the ideal schedule requires both

$$t_A \leq t_0 + D_A, \quad t_R \leq t_0 + D. \quad (5)$$

Under these conditions, the ideal holds are

$$\begin{aligned} \hat{e}_A - t_A &= D_A - (t_A - t_0), \\ \hat{e}_R - t_R &= D - (t_R - t_0). \end{aligned} \quad (6)$$

Consequently, the holds absorb changes in arrival time without moving the scheduled departures. If either packet arrives late, its release depends on when it arrives and when the queue can serve it. To follow both deadlines, we must admit the transaction, sustain each hold with enough blockers, and receive the acknowledgment before its earlier deadline, even when the response meets its budget.

Actual Release. For an admitted transaction whose packets arrive in time and whose holds last until their deadlines, write $e_A = \hat{e}_A + \varepsilon_A$ and $e_R = \hat{e}_R + \varepsilon_R$. These release tails include the time for remaining blockers to drain and for the held packets to be served and forwarded. Then

$$\begin{aligned} e_A - t_0 &= D_A + \varepsilon_A, \\ e_R - e_A &= \text{CLRT}_{\text{new}} + \varepsilon_R - \varepsilon_A. \end{aligned} \quad (7)$$

Consequently, the actual gap differs from its configured value by the difference between two release tails, neither of which the operator directly sets. Section VI-B3 evaluates the master-facing excess over the ACK deadline, which also includes path and capture effects.

Security Argument. Equation (7) removes the explicit arrival-time terms from the scheduled intervals. If the residual tails and path effects are independent of device execution, the released intervals no longer expose that execution time. However, this independence is an assumption to examine: queue state, late arrivals and an exhausted blocker budget can affect release, even though the deadlines themselves are request-anchored. We therefore evaluate both the concentration around the configured interval and the timing information retained by fixed and adaptive classifiers (Section VI-C). The argument motivates applying the schedule to other devices, but one device cannot establish that different device models become indistinguishable.

1) *Control Transactions:* To separate the forwarding of an OPERATE from its arrival at the switch, we also configure a set of command holds J . When the OPERATE arrives at T_0 , the switch selects a hold and schedules forwarding toward the relay at $T_0 + J$. Actual forwarding can trail this instant because of queue service. We configure the returning acknowledgment and response with the same offsets as the read lane:

$$\hat{e}_{\text{ack}} = T_0 + D_A, \quad \hat{e}_{\text{or}} = T_0 + D_A + \text{CLRT}_{\text{new}}. \quad (8)$$

Because both packets must reach the switch before these deadlines, their availability must account for the command hold and the relay's work after receiving the command, unlike a READ. Consequently, the largest admissible J must leave time for both the acknowledgment and response, not just for the response to meet D .

Security Argument. Under the same scheduling conditions, the released OPERATE interval is $O = e_{\text{or}} - e_{\text{ack}} = \text{CLRT}_{\text{new}} + \varepsilon_{\text{or}} - \varepsilon_{\text{ack}}$ at the switch boundary. The command hold does not appear explicitly in this expression. As such, we can configure different command holds under one return schedule, provided the packets are available in time and the residual release effects remain acceptable. The campaign records the configured set $\{2, 6, 12\}$ ms and the intervals observed at the master; it does not measure the selected J or physical actuation on each exchange.

2) *Latency and Timer Constraints: Delay and Coverage.* A larger response budget can accommodate slower responses, but it also increases the scheduled request-to-response interval. For an on-time response released after its deadline, the hold is $D - (t_R - t_0) + \varepsilon_R$, which is at most $D + \varepsilon_R$. This is a bound on its hold given a release tail, not a bound on every measured transaction. Changing the split between D_A and the configured CLRT keeps the response deadline fixed, but only splits that satisfy both availability conditions are usable. We evaluate this trade-off in Section VI-A.

Protocol Timers. Three endpoint timers constrain the settings. First, the master waits for a TCP acknowledgment before retransmitting its request. Second, the outstation waits for

acknowledgment of its response. Last, the relay waits for OPERATE after accepting SELECT. The last interval includes the wait for the SELECT response, the master’s next request, and the OPERATE hold. To choose a policy, the operator must leave room for these existing delays, network transit and release tails within each timer. These margins depend on the deployment, rather than on the configured budget alone.

RFC 6298 recommends a one-second lower bound on the retransmission timeout [26], but we check endpoint behavior rather than assume that recommendation is enforced. The SEL-751A specifies a default select-to-operate timeout of 1.0 s [27]. Section VI-B2 reports the observed transaction waits and the separate retransmission tests. Those measurements qualify the settings on this testbed; they do not guarantee negligible delay in every deployment.

Supported Holding Range. Blockers have a finite pass budget, so a configured deadline can exceed the hold that the implementation sustains. The control plane uses an estimated horizon H to restrict admission, and the policy sweep checks where the measured release stops following the configured deadline. Because this horizon is estimated rather than measured as a worst-case bound, we also check the supported range experimentally; settings beyond it may release packets early and lose timing protection.

C. Queue and P4 Realization

In Figure 4, we map the queueing idea to the switch pipeline. The parser exposes the fields used to identify a packet, based on which ingress chooses whether to forward it, hold it or process it as an internal blocker. The traffic manager stores these packets and selects which queue to serve. Consequently, the wait occurs in the queues, while the P4 program runs to completion on each arrival.

Ingress Decisions. To protect a fresh transaction, ingress records its identity and writes the request-anchored deadlines. Ingress directs the returning acknowledgment and response to their held queues, while an internal blocker is re-enqueued or dropped according to its identity, deadline and remaining pass budget. A released real packet instead takes the normal forwarding path. The evaluated implementation keeps one protected transaction in flight; a second request that cannot be admitted is forwarded without this timing protection.

Queue Scheduling. The ACK blocker, held ACK, response blocker and held response occupy four queues, in descending priority, on one internal port. Consequently, the response and ACK queues share a scheduler. OPERATE uses a separate internal port, with its blocker queue above its held-command queue, so ACK traffic cannot starve the command. Strict priority provides the hold only while the relevant higher-priority queues remain nonempty [22].

Loopback and Output. Packets served from either internal port bypass egress and return to ingress. This path lets blockers repeat their deadline checks without reaching an external link. After release, a real packet returns to ingress once and is forwarded through normal egress toward the master or outstation. Internal blockers add no traffic to those external

links. Other obfuscation mechanisms use external traffic for their own objectives [4], [11]; the waiting mechanism here uses the switch’s internal queues. The implementation details and the handling of retransmitted commands are described in Sections V and VI-E.

V. IMPLEMENTATION

To evaluate the framework, we implemented it on an Intel Tofino-1 switch and placed the switch between a DNP3 master and a physical protective relay. In this section, we describe the deadline and transaction state used by the queueing mechanism, then present the testbed.

A. Deadline and Transaction State

Section IV-C describes the queue and pipeline mapping. Here we describe the state that realizes those decisions. The program stores absolute deadlines on a 256 ns grid. Each blocker reservoir contains 64 packets, which test the relevant deadline on every internal return. ACK and response blockers use separate reservoirs, while their queues share one scheduler. A deadline ends blocker recirculation; packets already queued still require service before release.

Transaction State. A fresh protected request records the transaction and arms both return deadlines. A later acknowledgment cannot move those deadlines. The response enters its held queue even if it arrives late, and its release clears the read-lane transaction. Blocker tags allow ingress to reject tokens from an old transaction when they return. They do not remove the service already consumed by tokens waiting in a queue.

Control State. SELECT prepares the control lane, and an admitted OPERATE selects its command hold. Both lanes share the return-deadline registers, protecting one transaction at a time. The control lane retains the released OPERATE generation until the next SELECT and drops a matching retransmission. Consequently, it does not provide recovery of a command lost toward the relay (Section VI-E).

Deadline Selection. To vary return timing, the randomized image selects the offsets from a 16-entry range table indexed by an eight-bit random value. The selected offsets arm the existing deadline registers. The table uses the same random byte that selects J , so these choices can be correlated. A control-plane digest records the request identity and selected offsets. We match these records to captured requests to check the policy chosen for each exchange; the capture separately supplies the observed packet times.

Holding Limits. Blockers stop on expiry, stale state or exhaustion of their pass budget. The control plane uses a horizon estimate $H = 30.8$ ms to describe the supported hold, but this is not an elapsed-time limit directly enforced by the data plane. Packets outside the protected session and requests that cannot be admitted follow bypass paths. With timing disabled, the program forwards packets without scheduled holds, giving the Timing OFF case.

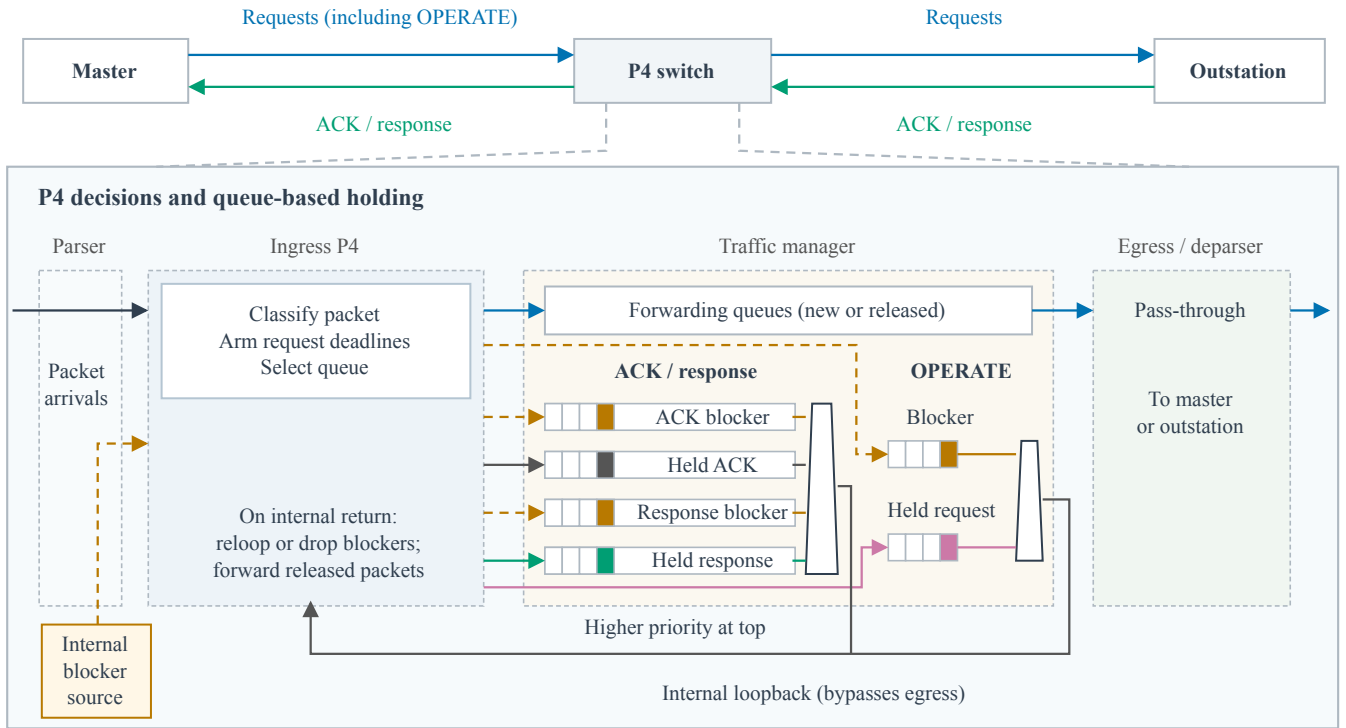


Fig. 4. Packet flow and its mapping to the P4 pipeline. Ingress classifies packets, arms request-anchored deadlines and selects queues; the traffic manager provides the wait. ACK and response queues share one strict-priority scheduler, while OPERATE requests use a separate scheduler. Internal loopback bypasses egress. Returning blockers are re-enqueued or dropped according to expiry; after the higher-priority queues drain, the released packet returns to ingress and follows the normal output path. Scheduled deadlines do not include the residual release tail.

B. Testbed

Switch Program. We used three compiled P4₁₆ [21] images of the same timing-obfuscation mechanism for the Tofino Native Architecture. The main 63,360-exchange campaign used the image that occupied all twelve ingress stages and six egress stages of one Tofino-1 pipeline, with 113 tables. After simplifying the control logic and removing unused pipeline work, the low-latency fixed-policy study used a second image on the same switch and the same Barefoot SDE 9.13.2 toolchain; that image occupies seven ingress match-action stages, zero egress match-action stages and 73 tables. The randomized image adds deadline selection and digest export, occupying seven ingress match-action stages, zero egress match-action stages and 76 tables. All three images implement the same request-anchored queueing abstraction and expose the same timing parameters, so we treat their timing measurements as separate implementation instances rather than as one mixed data set. One 25 Gb/s port connects the master and one 1 Gb/s port connects the relay. Two further ports are loopbacks without cables, one for the read lane and one for the control lane.

Control Plane. A control plane written in Python, around 900 lines in its entry module, brings up the ports and the queues, configures the packet generator, and writes the release policy into runtime tables and reads it back. An operator therefore changes the policy by writing table entries rather than loading a new program. The control plane refuses any

release offset at or above the smallest of the timer and horizon bounds it is given, which is 30 ms on our testbed.

Physical Device. The outstation is an SEL-751A feeder protection relay, one of the DNP3 device classes that Formby et al. fingerprint [14].

DNP3 Master. The master issues READ polls and select-before-operate controls through a guarded driver. Every control operation addresses an isolated control point, so no operation in the evaluation moves a breaker.

VI. EVALUATION

In this section, we evaluate whether the framework changes timing as configured (RO1), what delay it adds and how that delay interacts with protocol timers (RO3), and what timing information an adversary still learns (RO2). We first describe the measurements used to answer these questions.

Evaluation Cases. We compare two cases on the testbed of Section V. In *Timing OFF*, the switch forwards packets without scheduled holds. In *Obfuscated*, it applies the request-anchored schedule of Section IV. Both cases run the same switch program. To measure what the adversary observes, we timestamp packets on the master-facing link and compute request-to-ACK time, CLRT from ACK to response, and total request-to-response time. These observations do not provide switch-departure or relay-facing timestamps.

Datasets and Settings. We collected 63,360 DNP3 exchanges in 132 captures over 5.3 hours, organized into 22

paired runs with both cases and equal exchange counts in each case. To reduce confounding by drift, we interleaved Timing OFF and Obfuscated throughout the campaign. The Obfuscated configuration sets $D_A = 20$ ms and CLRT to 8 ms, giving a 28 ms response budget, and installs the command-hold codebook $\{2, 6, 12\}$ ms. Readbacks verify each block's configuration, but the per-exchange command hold is unobserved. Every request in the main campaign received a response, and no exchange was excluded from the transaction statistics.

To check whether timing follows a change in policy, we also collected 9,360 exchanges across 26 sweep settings: 23 release policies and three controls. Each setting has a control-plane readback. We estimate read-lane coverage from the Timing OFF request-to-response distribution and evaluate realized timing from Obfuscated exchanges, which are separate observations rather than repetitions of the same relay execution.

Finally, we collected a low-latency fixed-policy study with the seven-ingress-stage image described in Section V-B. This study contains 37,500 validated primary exchanges in 125 blocks: 25 policies, five repetitions per policy, and 100 READ, 100 SELECT and 100 OPERATE exchanges per repetition. Each block also includes two safety status polls outside the primary count. The capture statistics reported zero drops, and an independent packet-filter check found no TCP retransmissions. The grid uses configured $D_A \in \{5, 10, 15, 20\}$ ms and configured $\text{CLRT}_{\text{new}} \in \{1, 2, 4, 6, 8\}$ ms, plus two additional fixed policies at $(D_A, \text{CLRT}_{\text{new}}) = (4, 4)$ ms and $(12, 4)$ ms. It also repeats three reference settings: Timing OFF at $(20, 4)$ ms and Obfuscated at $(20, 4)$ ms and $(20, 8)$ ms. The grid policies use $J \in \{0.25, 0.5, 1\}$ ms for OPERATE; the three references use the historical $J \in \{2, 6, 12\}$ ms codebook. Every policy readback contained exactly the three expected codebook entries.

A. Effectiveness of Timing Replacement

To evaluate RO1, we ask whether the released interval concentrates around its configured value, whether the change differs from adding a constant delay, and where the schedule ceases to hold. We evaluate READ and SELECT together as the read lane, then examine OPERATE separately.

READ and SELECT. In Figure 5, we show the READ CLRT measured in both cases. Under Timing OFF, the interval spans 1.007 to 113.480 ms, whereas under Obfuscated, 99.94% of READ exchanges lie within 0.10 ms of the configured 8 ms. The READ median moves from 2.121 to 8.005 ms, while its interquartile range falls from 2.772 to 0.022 ms. SELECT follows the same schedule: its median moves from 2.041 to 8.005 ms and its interquartile range falls from 2.679 to 0.021 ms.

Replacement versus a Constant Shift. Figure 6(a) compares the measured READ distributions with a counterfactual that shifts Timing OFF by one constant to match the Obfuscated median. The shifted READ distribution retains its standard deviation of 2.889 ms, while the framework reduces it to 0.989 ms; SELECT falls from 2.267 to 0.017 ms. These

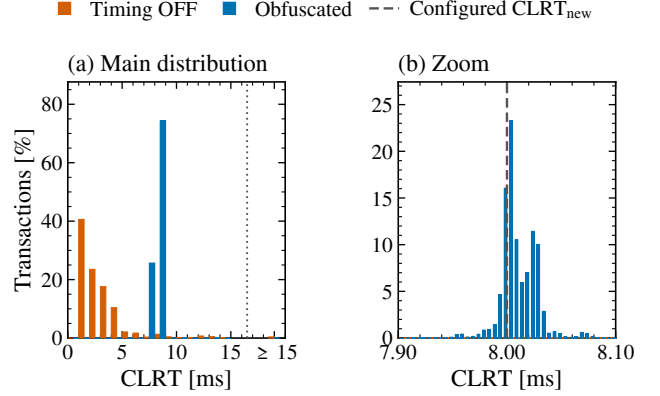


Fig. 5. READ CLRT on the master-facing link, as a percentage of each case's own transactions. (a) 1 ms bins, with everything at or above 15 ms in the category on the right. (b) 0.005 ms bins within 0.10 ms of the configured value, dashed. This window contains 0.09% of Timing OFF and 99.94% of Obfuscated transactions, using each case's entire sample as the denominator.

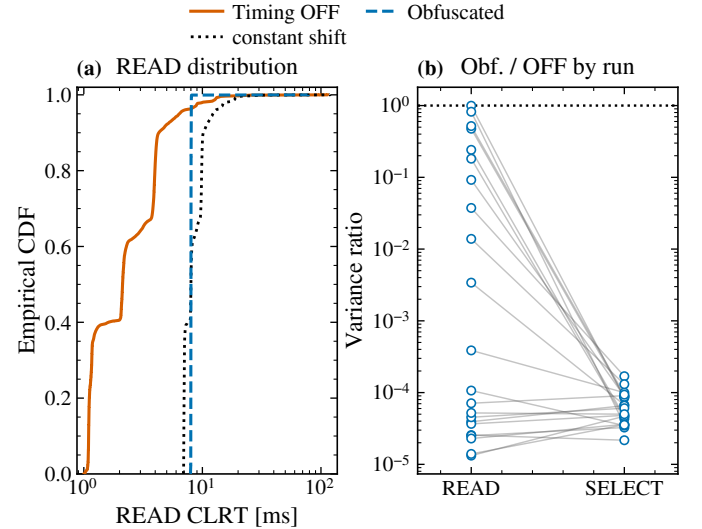


Fig. 6. Replacement versus a constant shift, measured on the master-facing link. (a) READ empirical CDFs, including all observations. The shifted curve is a counterfactual obtained by adding one constant to Timing OFF samples to match the Obfuscated median. (b) Obfuscated-to-Timing OFF sample-variance ratios for each paired run; connecting lines join READ and SELECT from the same run. A constant shift preserves variance, giving a ratio of one.

reductions agree with the variable holds in Section IV-B: earlier arrivals wait longer, so the change is not simply the same added delay on every packet.

In Figure 6(b), we show the READ and SELECT variance ratios for each paired run, with a line joining the two operations; both ratios remain below one in every run. Under the stronger comparison against every Timing OFF run, 21 of the 22 Obfuscated READ runs also have lower variance. Because the remaining run contains large deviations from the configured interval, we retain its observations in the plots, including the long tail discussed below.

Control Lane. For OPERATE, the master-facing CLRT median moves from 2.929 to 8.000 ms, and the interquartile range falls from 2.729 to 0.006 ms. The measured interval follows the return schedule in (8) under the installed command-hold codebook. Note that we evaluate timing at the master, since neither the selected J nor the relay-facing release was observed; the result is not a comparison between individual holds.

Programmable Policy. In Figure 7(b), we vary the configured CLRT from 2 to 26 ms at a fixed response budget of 28 ms. Measured CLRT medians remain within 6 μ s of the setting, while median request-to-response time stays between 28.106 and 28.108 ms. Consequently, within these tested settings, an operator can change the interval without materially changing the total response delay. This is because changing the split between the two offsets leaves the response deadline fixed, provided both packets are available before their respective deadlines.

The sweep also shows a holding limit. The measured request-to-ACK median follows a 30 ms deadline to 30.107 ms, with CLRT still at 3.999 ms for its configured 4 ms. At larger tested deadlines, ACK timing saturates near 31.07 ms, and CLRT reaches 9.034 ms at a 36 ms setting. Consequently, the sweep identifies where this implementation stops following the schedule under the tested traffic, while the control-plane horizon remains an estimate rather than a measured worst-case bound.

Coverage and Late Responses. Figure 7(a) shows the Timing OFF request-to-response distribution for READ and SELECT. A 28 ms budget covers 99.94% of these exchanges; 17 of 29,040 exceed it. This is a master-facing estimate of response-budget coverage, not a direct test of both switch-side availability conditions. In the separate Obfuscated case, 15 of 26,400 READ intervals depart from the configured value by more than 1 ms, and the maximum CLRT is 78.443 ms. The long tail remains visible in the Obfuscated distribution. Because the two cases contain different exchanges, this comparison cannot attribute individual departures to the 17 Timing OFF observations.

B. Performance and Release Accuracy

To evaluate RO3, we measure the added transaction delay and check whether the observed waits trigger retransmission or rejection of a control command. We then examine the excess

over the configured ACK deadline to qualify how closely measured timing follows the schedule.

1) *Transaction Cost: Bandwidth.* Each capture carries 1,448 frames and 130,708 bytes on the master-facing link in both cases, or 3.017 frames and 272.308 bytes per exchange. Consequently, we observe no added external traffic on the captured link, consistent with the mechanism in Figure 4, where blockers remain on internal loopback paths.

Latency. In Figure 7(c), we show request-to-response time in both cases. The difference between the case medians is 25.426 ms for READ, 25.482 ms for SELECT and 24.652 ms for OPERATE. These differences between case medians remain below the 28 ms response budget, consistent with holding responses until the configured deadline; they are not per-packet hold measurements. To assess this cost on our testbed, we check the observed waits against endpoint behavior below; other deployments must also satisfy their application latency requirements.

2) *Impact on Protocol Timers: Retransmission.* Neither case contains a TCP retransmission on the master-facing captures. Under Obfuscated, the longest observed request-to-ACK and request-to-response waits are 20.19 and 98.55 ms. In a separate test that withheld every acknowledgment of one request, the master retransmitted after 200.8 ms. The longest campaign ACK wait is about one tenth of that observed timeout. A separate loss trace recorded a repeated response after 2.969 s; this is one observed repetition interval, not a lower bound on all relay retransmissions. We use these observations to assess the tested waits, without treating campaign maxima as bounds on future traffic.

Select Validity. The master issues OPERATE a median of 0.354 ms after receiving its SELECT response. Because the master waits for that response, its hold consumes part of the select-to-operate interval, as does the subsequent command hold. All 5,280 Obfuscated SELECT and OPERATE exchanges returned a success status, supporting successful protocol completion at the tested settings. Physical actuation and relay-facing command multiplicity were not measured.

3) *Release Accuracy and Blocker Drain: Measured Excess.* To examine release accuracy, we subtract configured D_A from the measured request-to-ACK interval. In Figure 8, we show this master-facing excess, which includes the ACK release tail together with request transit, return transit and capture effects. Because we have no switch-departure timestamps, it does not isolate the drain of the blocker queue or measure the response release tail.

At the campaign setting, the median excess is 0.101 ms for READ and SELECT and 0.107 ms for OPERATE; the central 99% of values lie between 36 and 127 μ s. Across configured ACK deadlines from 4 to 30 ms, the median stays between 0.098 and 0.108 ms. Consequently, the measured excess remains small across the tested settings even as the configured deadline increases, indicating close tracking of the schedule in this range. Direct blocker-drain measurements would be needed to separate the contribution of the queues from the other effects. Once release precedes the configured

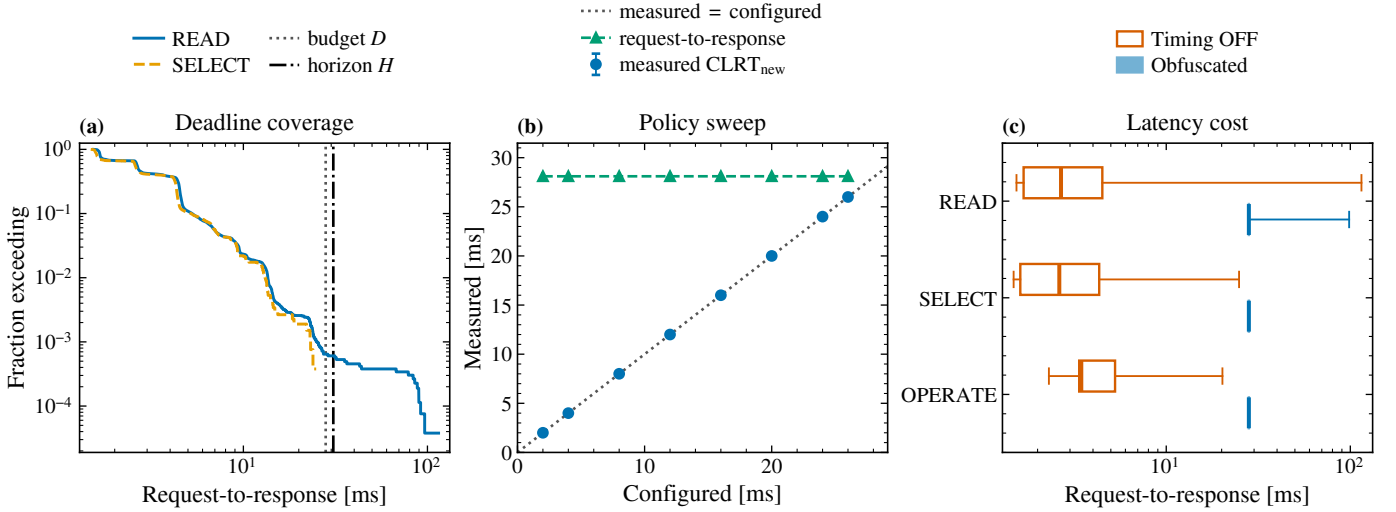


Fig. 7. Release policy and cost. (a) The Timing OFF request-to-response time the budget must cover, for the read lane only. (b) Measured against configured interval at a fixed budget $D = 28$ ms. (c) Request-to-response time per operation, in both cases.

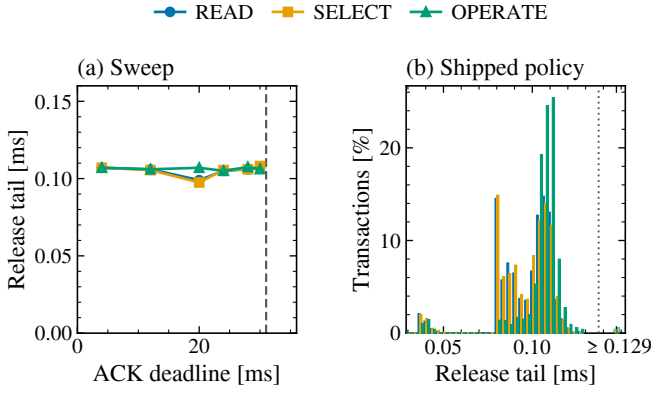


Fig. 8. Master-facing excess over the configured ACK deadline, used as a release-tail proxy. (a) Against the configured deadline, one series per operation. Past the dashed line, measured ACK timing stops following the setting; the excess no longer represents post-deadline release. (b) Distribution at the campaign policy, including every value at or above 0.129 ms in the rightmost category. Percentages use each operation’s full sample. The plotted excess includes path and capture effects and does not directly measure blocker-drain time.

deadline, the same subtraction no longer measures a post-deadline excess.

C. Residual Timing Information

Adversary Models. To evaluate RO2, we classify READ, SELECT and OPERATE using a random forest on CLRT alone or on CLRT and request-to-ACK time. Motivated by timing fingerprinting [14], we test transaction-class information for one device by holding out one paired run at a time, with three-class chance balanced accuracy of 0.333. The fixed adversary trains on Timing OFF and is applied unchanged; the adaptive adversary retrains on Obfuscated traffic. We also estimate

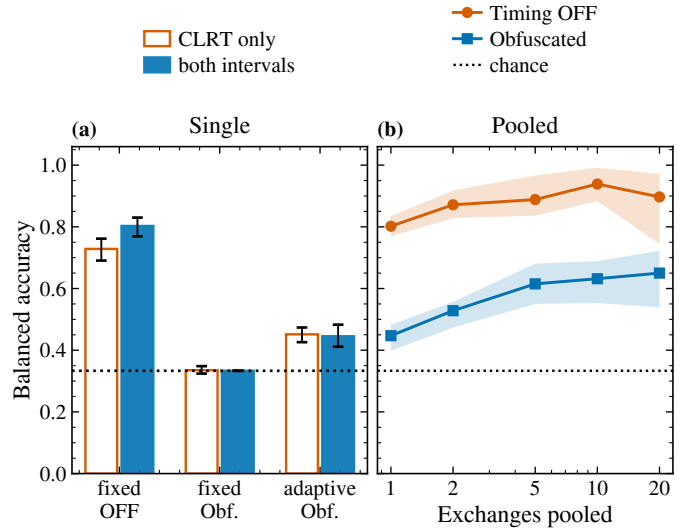


Fig. 9. Fingerprinting the three operations, one grouped run held out at a time. (a) Balanced accuracy by adversary; the fixed adversary is trained on Timing OFF traffic and applied unchanged, the adaptive one is retrained on obfuscated traffic. (b) What pooling several exchanges of one operation buys the adaptive adversary, from both intervals. Whiskers and bands span the 22 runs, which is within-campaign variability and not a confidence interval.

CLRT-class mutual information and compare it with a null from 1,000 within-run label permutations.

Fixed Adversary. In Figure 9(a), the fixed adversary reaches 0.728 balanced accuracy under Timing OFF using CLRT and 0.803 using both intervals. After applying the classifier unchanged to Obfuscated traffic, we observe accuracy of 0.335 and 0.333, close to chance. CLRT-class mutual information falls from 0.353 to 0.043 bits, but the remaining estimate exceeds its permutation null ($p = 0.001$). Consequently, the failure of this fixed classifier does not imply that the released timing contains no class information.

Adaptive Adversary. After retraining, accuracy is 0.451

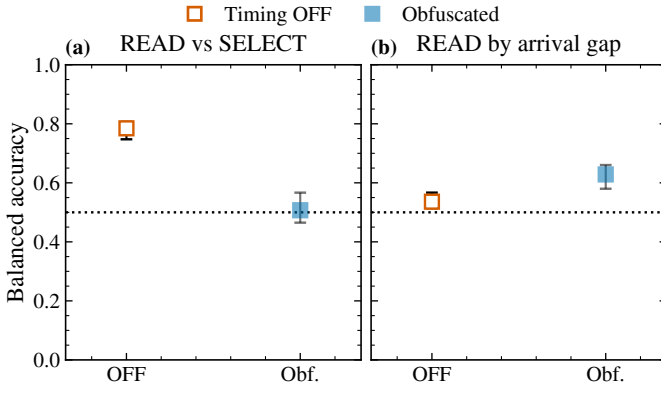


Fig. 10. Residual timing information in two binary classification tasks on master-facing intervals: (a) READ versus SELECT; (b) READ split at each case’s median preceding gap, measured from the previous response in the same capture. Both use the CLRT and request-to-acknowledgment interval, training separately in each case and holding out one run at a time. Points show mean balanced accuracy; whiskers span the held-out runs, not confidence intervals. The dotted line marks chance (0.5). These are observational comparisons, not a controlled intervention on request spacing; the gap analysis omits the first exchange in each capture, whose preceding gap is undefined.

from CLRT alone and 0.445 from both intervals. The single-exchange classifier using both intervals performs best on OPERATE, with recall 0.689, compared with 0.411 for READ and 0.235 for SELECT. Figure 9(b) examines a separate attack that pools consecutive exchanges of the same operation within a capture. To construct a signature from each non-overlapping group, we use the mean, standard deviation, minimum and maximum of each interval. With twenty exchanges, balanced accuracy reaches 0.650 under Obfuscated, compared with 0.897 under Timing OFF, showing that pooling strengthens the evaluated adaptive attack.

READ versus SELECT. To examine two operations handled by the same queues, we train and test a binary classifier separately in each case. Figure 10(a) shows that accuracy from both intervals falls from 0.785 under Timing OFF to 0.507 under Obfuscated, against chance of 0.5. The Obfuscated medians are identical: 8.005 ms CLRT and 20.101 ms request-to-ACK time. The shared schedule is consistent with this reduction in timing differences, although a near-chance result for one classifier does not establish the absence of all execution-time information.

Request Spacing. Figure 10(b) examines READ alone, split at each case’s median gap from the preceding response to the next request. We omit the first exchange in each capture, whose preceding gap is undefined. The Obfuscated groups differ by 31 μ s in median gap, and the classifier separates them at 0.628 accuracy, compared with 0.536 under Timing OFF. Because the gap defines the groups but is not supplied as a classifier feature, this result indicates that the released intervals retain information associated with request spacing, even within one operation.

The traffic pattern also differs across operations: an Obfuscated READ follows its predecessor by a median of 20.505 ms, whereas OPERATE follows SELECT by 0.354 ms. We expect

request spacing and release state to contribute to the adaptive result, but this observational comparison does not isolate either as the cause. The design argument remains conditional on the residual effects identified in Section IV-B.

D. Low-Latency Fixed Policies

The 28 ms campaign policy leaves room to reduce delay, so we measured a fixed grid of smaller ACK deadlines and configured CLRT values on the simplified implementation. Figure 11 shows the resulting request-to-response times. The lowest-median-latency setting, $(D_A, \text{CLRT}_{new}) = (5, 1)$ ms, has a worst-operation median of 6.109 ms and a worst-operation p99 of 15.446 ms. The repeated (20, 8) ms reference has a worst-operation median of 28.109 ms, so the measured worst-operation median falls by 22.001 ms. Relative to the Timing OFF reference, the same (5, 1) ms point adds 3.549 ms at the median, whereas the (20, 8) ms reference adds 25.549 ms.

The grid also shows why configured values and measured values must remain separate. The configured response budget is $D_A + \text{CLRT}_{new}$, and the realized data-plane values are quantized to a 256 ns grid. The measured median includes the endpoint path and capture effects, so it is about 0.106–0.109 ms above the configured sum when the schedule is followed. At the low end, the p99 can exceed the configured median budget, consistent with the conditional availability requirement in Section IV-B; for example, the (5, 1) ms point has a 15.446 ms worst-operation p99 even though its median is 6.109 ms. These measurements therefore identify lower-cost timing configurations for this workload, but they do not by themselves select a final security point against an adaptive classifier.

Classifier Screen. Figure 12 compares the latency grid with a classifier screen over the 24 protected policies. For each policy, we evaluated READ versus SELECT and the three-class READ/SELECT/OPERATE task on protected test traffic. The fixed attacker trains on Timing OFF traffic and is then applied to protected traffic; the adaptive attacker trains and tests on protected traffic with grouped held-out acquisition repetitions. We tested random forests, logistic regression and radial-basis-function support-vector-machine models, with nested grouped tuning restricted to the training repetitions, and pooling levels of 1, 5 and 20 same-operation exchanges. The screen reports the largest balanced accuracy over the fixed and adaptive attacks, feature families, models and pooling levels. We resample acquisition repetitions together across attacks and use the maximum centered bootstrap deviation to form an upper bound. We use 97.5% per task for an approximate 95% combined screen across the two tasks. A policy passes only if the simultaneous upper bound is at most chance plus 0.05, which is 0.550 for READ versus SELECT and 0.383 for the three-class task. This is a development screen, not an independent confirmation experiment.

For the lowest-median-latency setting, $(D_A, \text{CLRT}_{new}) = (5, 1)$ ms, the maximum ACK/response balanced accuracy is 0.600 for READ versus SELECT and 0.493 for the three-

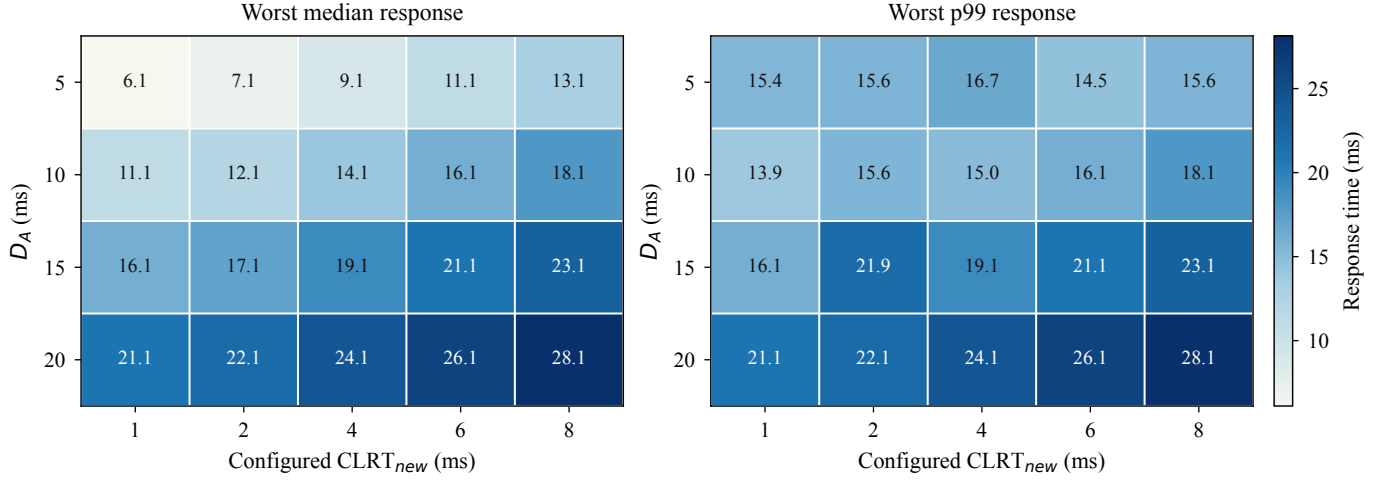


Fig. 11. Low-latency fixed-policy grid on the seven-ingress-stage implementation. Each cell reports the largest statistic across READ, SELECT and OPERATE on the master-facing request-to-response time. The left panel reports the worst median, and the right panel reports the worst p99, using a shared color scale.

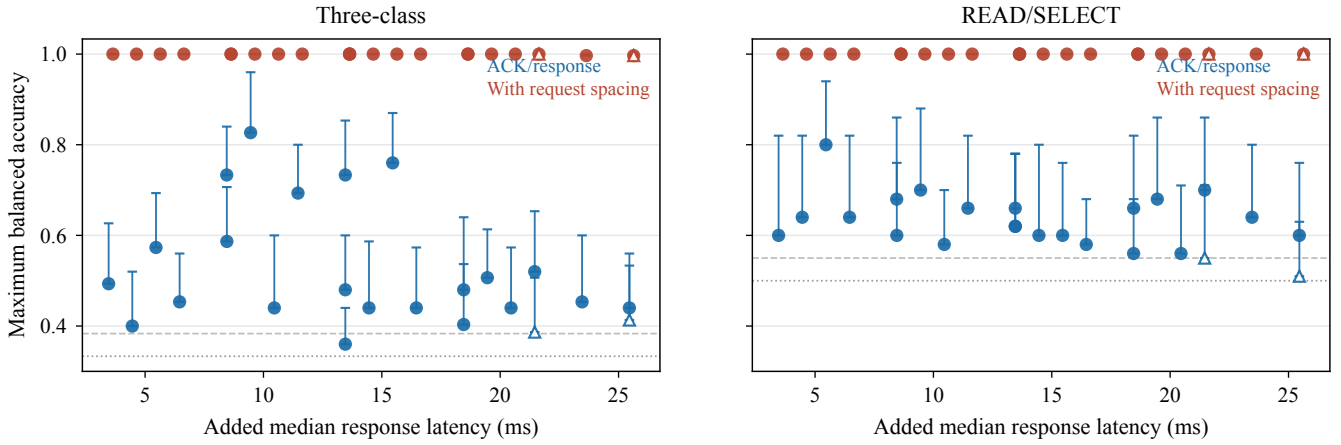


Fig. 12. Latency and classifier screen for the low-latency fixed-policy study. The x-axis is added median request-to-response time relative to the Timing OFF reference, taking the worst operation for each task. Points show the maximum evaluated balanced accuracy on protected test traffic; whiskers show the simultaneous development upper bound from five acquisition repetitions. The dotted horizontal lines mark chance, and the dashed lines mark the chance-plus-0.05 acceptance thresholds. Filled circles are search-codebook policies, and open triangles are historical-codebook references. ACK/response uses request-to-ACK and CLRT features. With request spacing also gives the attacker request-cadence features, which test harness cadence in addition to device timing.

class task. The corresponding three-class upper bound is 0.627, which exceeds the acceptance threshold. Pooling limits the held-out evidence: at pool size 20, each test repetition contributes only five same-operation signatures per class. These small groups widen the uncertainty from the five acquisition repetitions. Other policies yield higher observed accuracy through the same ACK/response features; at (10, 2) ms, the three-class attack reaches 0.827 balanced accuracy. When request spacing is included, the strongest evaluated attack reaches 1.000 balanced accuracy across the protected grid. That result reflects the same-session acquisition cadence available to the attacker; it should not be read as device-model identification from obfuscated response timing alone. Consequently, none of the 24 protected policies passes the strict development screen, even though the grid identifies policies with much lower measured latency than the 28 ms reference.

E. Limitations

Devices and Workloads. We evaluate one SEL-751A, one Tofino-1 and one testbed workload. The 22 paired runs in the main campaign describe within-campaign variability, and their held-out-run ranges are not confidence intervals. The low-latency screen uses approximate grouped-bootstrap confidence bounds from five same-session acquisition repetitions, so it does not provide independent confirmation. Multi-device measurements, such as those used for fingerprinting by Formby et al. and Lin et al. [14], [20], are needed to test device-model distinguishability. Because the implementation protects one transaction at a time, requests that cannot be admitted bypass timing protection, which limits its coverage under concurrent traffic.

Residual Information. The results characterize the evaluated random forests, logistic regression and radial-basis-function support-vector-machine models and timing features. Adaptive accuracy remains above chance, and packets that miss their release deadlines can retain arrival-time variation. Neither the near-chance READ/SELECT result nor the observed association with request spacing establishes that all device information has been removed. In the low-latency screen, request-spacing features also expose same-session harness cadence, so those attacks qualify the measurement procedure as well as the timing mechanism.

Unobserved Internal Events. Master-facing captures do not reveal the selected command hold, relay-facing release, command multiplicity or physical actuation. They also do not directly measure blocker-drain time. Configuration readbacks establish installed settings; they do not supply these missing event timestamps or turn the horizon estimate into a measured bound.

Retransmitted Commands. The control lane retains a released OPERATE generation until the next SELECT and drops a matching retransmission. Consequently, a command lost toward the relay cannot be repaired by its own retransmission, even though exchanges without that loss completed successfully in the campaign. We leave support for this recovery to future work.

VII. RELATED WORK

Device Fingerprinting. Devices can be identified from what they emit, without access to the payload. Kohno et al. fingerprint a remote host from the skew of its clock in TCP timestamps [28], and GTID identifies a device and its type from the timing signature of its packets [29]. These works define the threat we address, and each of them reads a different observable. Compared to them, we target the interval that the cross-layer response-time method of Formby et al. classifies by, and we replace that interval on the master-facing link.

ICS and Power-System Fingerprinting. Formby et al. brought fingerprinting to control systems with the cross-layer response time and the physical operation time of DNP3 outstations [14], and Gu et al. added device physics as a feature [30]. Pospisil et al. identify PLC models passively from the time delay in their dataflow [17], which shows that execution time still identifies industrial devices under other protocols. The reconnaissance these techniques serve enables false data injection and process-control attacks [24], [31]. Consequently, the interval our framework replaces is the observable this line of work relies on.

Anti-Reconnaissance in Power Grids. The defenses closest to ours change what the traffic says and which nodes it reaches. DefRec virtualizes physical functions and crafts decoy data for virtual nodes [20], RAINCOAT randomizes data acquisition and injects decoy measurements [32], and a companion testbed evaluates such approaches on grid infrastructure [33]. These approaches are effective against reconnaissance of network and physical configurations. However, they build their protection above the packets an outstation sends, so the execution time of a real device still travels with its answer. Our framework, on the other hand, works below the semantics they reshape and changes when the wire carries that answer, which makes the two complementary.

General Traffic Obfuscation. Traffic morphing transforms the packet-size distribution of one class into another [3], while Dyer et al. show that per-packet padding and fragmentation still leave exploitable features [4]. Tamaraw fixes packet size and rate at a bandwidth cost [8], and WTF-PAD pads inter-packet gaps adaptively [10]. These defenses are effective against the flow-level features they target, and the attacks that follow them have become correspondingly strong [5], [7]. The same size-and-timing channel identifies smart-home devices under encryption, where shaping is the countermeasure [34], and Pacer and NetShaper shape the traffic of a tenant to a schedule that bounds side-channel leakage in the cloud [35], [36]. Pacer and NetShaper each shape from a host or a hypervisor that can add cover traffic, and the smart-home setting assumes an encrypted payload. Compared to these settings, ours carries a plaintext protocol whose leaking feature is one interval inside a single request-response exchange, and no endpoint can be modified to help. Consequently, our framework adds no byte and runs entirely in the network.

Programmable Data-Plane Traffic Transformation. P4 defines the programmable parser and match-action model

we use [21]. PIFO defines programmable scheduling at line rate [37], and SP-PIFO approximates it with the strict-priority queues of today’s switches [22], which is the primitive our blocker queues use. Ditto pads packets and injects chaff to a fixed pattern at line rate, and it carries up to 70 Gbps of production traffic on a 100 Gbps link [11]. Minos morphs and schedules traffic on a programmable switch at low cost [12], and Securitas combines fragmentation and insertion across several data planes [13]. NetHide obfuscates topology in the data plane against link-flooding reconnaissance [38]. Closest to our setting, Hu et al. use programmable switches to enhance the security of industrial protocols [39]. These systems reshape which packets a link carries and how large they are. Our framework, on the other hand, addresses one interval inside a transaction, and it holds packets rather than adding or reshaping them.

DNP3 Timing and Security. East et al. catalogue attacks on DNP3 and the absence of encryption that makes its semantics visible [23], and specification-based and state-based intrusion detection has been adapted to DNP3 [25], [40]. These defenses inspect what a packet says, and none of them changes the timing an adversary sees. On the control path, Formby et al. read the physical operation time from an event timestamp inside the application payload [14], which is a different channel from the packet timing our framework replaces.

Protecting the Framework Itself. Like any in-network defense, the framework adds a switch to what an operator must protect, and an adversary that compromises the switch sees every hold. However, the framework makes light use of the switch. One pipeline program holds the packets of three function codes and forwards all others unchanged, no controller sits in the forwarding path, and the control plane reads back every policy it installs. Checking this program and its policy is a smaller task than hardening the legacy outstations behind it, which often cannot be patched.

VIII. CONCLUSION

We propose an in-network framework that holds the packets carrying the timing of an ICS device and releases them at deadlines computed from the arrival of the request. For packets available before their deadlines, the schedule can absorb arrival-time variation. Queue service, late arrivals and the supported holding range determine how closely the observed intervals follow the configured values. The framework runs without encryption and leaves the master, the outstation and DNP3 unchanged.

On a physical SEL-751A relay, CLRT medians for READ, SELECT and OPERATE approach the configured 8 ms, and the READ interquartile range falls from 2.772 to 0.022 ms. A classifier trained before deployment falls from 0.803 balanced accuracy to chance, while an adaptive classifier retains 0.445 accuracy and reaches 0.650 when pooling twenty exchanges. Median READ request-to-response time rises to 28.107 ms, with no retransmission observed in the campaign. The residual tests associate timing information with request spacing but do not isolate its cause.

In future work, we will evaluate the framework on outstations from several vendors, and we will investigate how blocker-queue state contributes to release timing.

OPEN SCIENCE

We reproduce every number and every campaign figure in this paper from the raw captures with one script, in a pinned environment archived with the extraction and analysis code. A hash manifest covers each published artefact, and a publication gate compares a fresh rebuild against what the paper reports, so a discrepancy fails rather than passing silently. We also publish the switch program that ran, with its compilation record. The firmware of the relay and its vendor documentation are not ours to distribute. On acceptance, we release the captures, the analysis pipeline and the switch program under an open license.

ETHICS CONSIDERATIONS

This work required no human subjects and no personal data. We ran every experiment on a laboratory testbed we own, namely one SEL-751A protective relay, one Intel Tofino-1 switch and one master host, on an isolated network with no connection to any operational system. We contacted no production grid, no utility network and no third-party device.

The relay is physical equipment whose outputs can operate a breaker. Every select-before-operate exchange in the evaluation addresses isolated control points through a guarded driver. No control operation in this paper moved a breaker or energized a circuit, and each OPERATE is a protocol exchange the relay accepted and acknowledged.

The framework is defensive. It removes a timing feature that an adversary uses for reconnaissance, and it neither creates a new attack nor discloses a vulnerability in the relay, the switch or the DNP3 protocol.

REFERENCES

- [1] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the cyber attack on the Ukrainian power grid," Electricity Information Sharing and Analysis Center (E-ISAC) and SANS Institute, Defense Use Case, Mar. 2016, tLP: White, 18 March 2016.
- [2] R. Langner, "Stuxnet: Dissecting a cyberwarfare weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.
- [3] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2009.
- [4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2012, pp. 332–346.
- [5] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2018, pp. 1928–1943.
- [6] T. Wang and I. Goldberg, "Walkie-talkie: An efficient defense against passive website fingerprinting attacks," in *Proc. 26th USENIX Security Symposium*, 2017, pp. 1375–1390.
- [7] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website fingerprinting at internet scale," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.
- [8] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 227–238.
- [9] W. Wang, M. Motani, and V. Srinivasan, "Dependent link padding algorithms for low latency anonymity systems," in *Proc. 15th ACM Conf. on Computer and Communications Security (CCS)*, 2008.
- [10] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an efficient website fingerprinting defense," in *Proc. European Symp. on Research in Computer Security (ESORICS)*, 2016, pp. 27–46.
- [11] R. Meier, V. Lenders, and L. Vanbever, "Ditto: WAN traffic obfuscation at line rate," in *Proc. Network and Distributed System Symposium (NDSS)*, 2022.
- [12] Z. Wang *et al.*, "Minos: A lightweight and dynamic defense against traffic analysis in programmable data planes," in *Proc. USENIX Annual Technical Conference (ATC)*, 2025.
- [13] G. Xie *et al.*, "Securitas: Defending against traffic analysis attacks with flexible in-network obfuscation," in *Proc. 23rd USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2026.
- [14] D. Formby, P. Srinivasan, A. M. Leonard, J. D. Rogers, and R. A. Beyah, "Who's in control of your control system? Device fingerprinting for cyber-physical systems," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.
- [15] C. Shen, C. Liu, H. Tan, Z. Wang, D. Xu, and X. Su, "Hybrid-augmented device fingerprinting for intrusion detection in industrial control system networks," *IEEE Wireless Communications*, vol. 25, no. 6, pp. 26–31, 2018.
- [16] C. M. Ahmed, M. Ochoa, J. Zhou, and A. Mathur, "Scanning the cycle: Timing-based authentication on PLCs," in *Proc. ACM Asia Conf. on Computer and Communications Security (ASIACCS)*, 2021, pp. 886–900.
- [17] O. Pospisil and R. Fudjak, "Identifying industry devices via time delay in dataflow," in *Proc. 13th Int. Conf. on Communication and Network Security (ICNS)*, 2023, pp. 173–177.
- [18] IEEE, "IEEE standard for electric power systems communications—distributed network protocol (DNP3)," IEEE Std 1815-2012, 2012, iEEE Std 1815-2012.
- [19] W. Eddy, "Transmission control protocol (TCP)," RFC 9293, Internet Engineering Task Force, 2022.
- [20] H. Lin, J. Zhuang, Y.-C. Hu, and H. Zhou, "DefRec: Establishing physical function virtualization to disrupt reconnaissance of power grids' cyber-physical infrastructures," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2020.
- [21] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.
- [22] A. G. Alcoz, A. Dietmüller, and L. Vanbever, "SP-PIFO: Approximating push-in first-out behaviors using strict-priority queues," in *Proc. 17th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 59–76.
- [23] S. East, J. Butts, M. Papa, and S. Sheno, "A taxonomy of attacks on the DNP3 protocol," in *Critical Infrastructure Protection III (IFIP AICT, Vol. 311)*. Springer, 2009, pp. 67–81.
- [24] Y. Liu, P. Ning, and M. K. Reiter, "False data injection attacks against state estimation in electric power grids," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2009, pp. 21–32.
- [25] H. Lin, A. J. Slagell, C. Di Martino, Z. Kalbarczyk, and R. K. Iyer, "Adapting Bro into SCADA: Building a specification-based intrusion detection system for the DNP3 protocol," in *Proc. 8th Annual Cyber Security and Information Intelligence Research Workshop (CSIIRW)*, 2013.
- [26] V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing TCP's retransmission timer," RFC 6298, Internet Engineering Task Force, 2011.
- [27] Schweitzer Engineering Laboratories, *SEL-751A Feeder Protection Relay Instruction Manual*, 2013, pM751A-01-NB, 20130329 printing; settings table, STIME01 select/operate time-out.
- [28] T. Kohno, A. Broido, and K. C. Claffy, "Remote physical device fingerprinting," *IEEE Trans. on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005.
- [29] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A technique for physical device and device type fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 5, pp. 519–532, 2015.
- [30] Q. Gu, D. Formby, S. Ji, H. Cam, and R. Beyah, "Fingerprinting for cyber-physical system security: Device physics matters too," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 49–59, 2018.
- [31] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks against process control systems: Risk assessment, detection, and response," in *Proc. 6th ACM Symp. on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366.
- [32] H. Lin, Z. T. Kalbarczyk, and R. K. Iyer, "RAINCOAT: Randomization of network communication in power grid cyber infrastructure to mislead attackers," *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 4893–4906, 2019.
- [33] H. Lin, B. Shrestha, and Y.-C. Hu, "Cyber-physical testbed: Case study to evaluate anti-reconnaissance approaches on power grids' cyber-physical infrastructures," in *Proc. Workshop on Learning from Authoritative Security Experiment Results (LASER)*, 2020.
- [34] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the smart home private with smart(er) IoT traffic shaping," *Proceedings on Privacy Enhancing Technologies (PoPETs)*, vol. 2019, no. 3, pp. 128–148, 2019.
- [35] A. Mehta, M. Alzayat, R. De Viti, B. B. Brandenburg, P. Druschel, and D. Garg, "Pacer: Comprehensive network side-channel mitigation in the cloud," in *Proc. 31st USENIX Security Symposium*, 2022.
- [36] A. Sabzi, R. Vora, S. Goswami, M. Seltzer, M. Lécuyer, and A. Mehta, "NetShaper: A differentially private network side-channel mitigation system," in *Proc. 33rd USENIX Security Symposium*, 2024.
- [37] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, A. Agrawal, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, "Programmable packet scheduling at line rate," in *Proc. ACM SIGCOMM Conference*, 2016, pp. 44–57.
- [38] R. Meier, P. Tsankov, V. Lenders, L. Vanbever, and M. Vechev, "NetHide: Secure and practical network topology obfuscation," in *Proc. 27th USENIX Security Symposium*, 2018, pp. 693–709.
- [39] Z. Hu, H. Lin, L. Waind, Y. Qu, G. Chen, and D. Jin, "Industrial network protocol security enhancement using programmable switches," in *Proc. IEEE Int. Conf. on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, 2023.
- [40] I. N. Fovino, A. Carcano, T. D. L. Murel, A. Trombetta, and M. Masera, "Modbus/DNP3 state-based intrusion detection system," in *Proc. 24th IEEE Int. Conf. on Advanced Information Networking and Applications (AINA)*, 2010, pp. 729–736.